in the constructor. Similarly, `NetworkClient` has an *explicit* dependency on the `EmailServerSettings` class.

TIP Generally speaking, any dependencies in your code should be explicit, not implicit. Implicit dependencies are hard to reason about and difficult to test, so you should avoid them wherever you can. DI is useful for guiding you along this path.

DI aims to solve the problem of building a dependency graph by inverting the chain of dependencies. Instead of the `RegisterUser` handler creating its dependencies manually, deep inside the implementation details of the code, an already-created instance of `EmailSender` is passed as an argument to the `RegisterUser` method.

Now, obviously *something* needs to create the object, so the code to do that has to live somewhere. The service responsible for providing the instance is called a *DI container* or an *IoC container*, as shown in figure 8.2.

DEFINITION The *DI container* or *IoC container* is responsible for creating instances of services. It knows how to construct an instance of a service by creating all its dependencies and passing them to the constructor. I'll refer to it as a *DI container* throughout this book.